

EXPERIMENT 1

Student Name: KUSHAGRA VASHISTHA
Branch: AIT-CSE-AIML
24AIT_KRG1_B Semester: 5
Subject Name: ADBMS

UID: 24BAI70915
Section/Group:

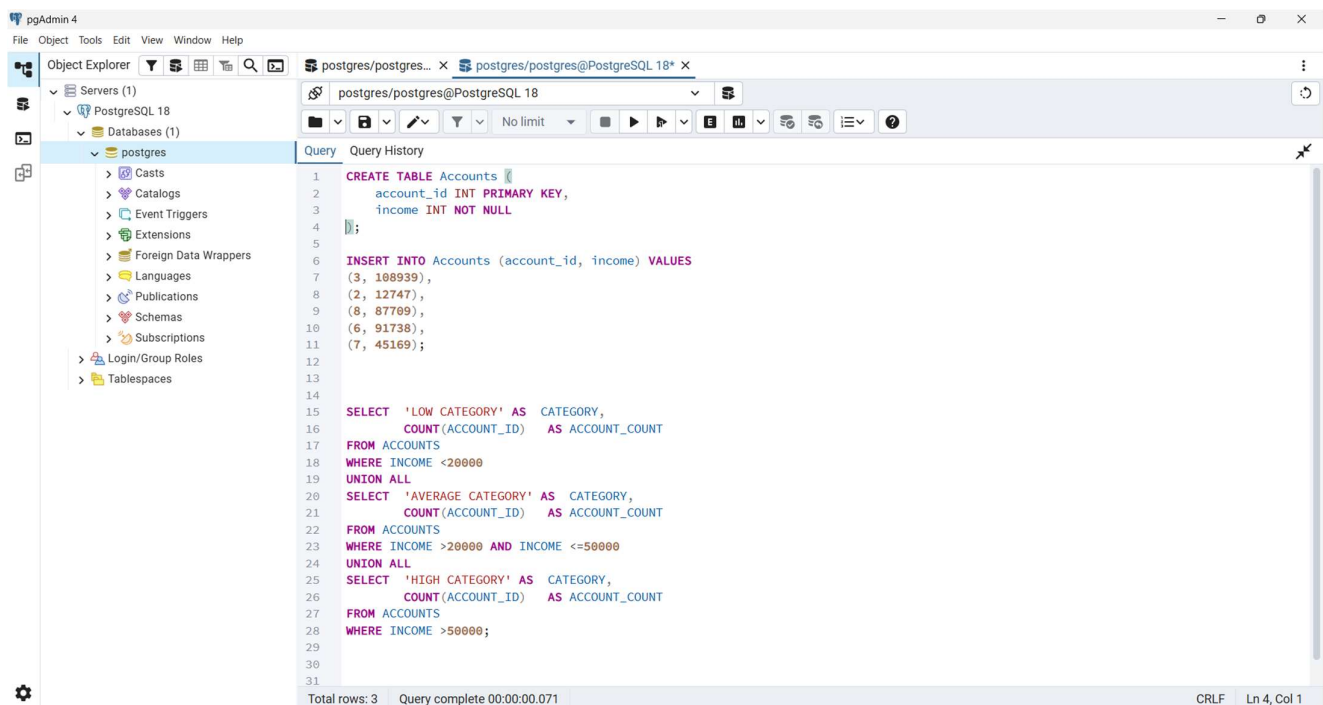
Subject Code: 24CSP-310

QUESTION A):

Write an SQL solution to calculate the total number of bank accounts belonging to each salary category. The classification categories are:

1. "Low Salary": All monthly salaries strictly less than \$20,000.
2. "Average Salary": All monthly salaries in the inclusive range [\$20,000, \$50,000].
3. "High Salary": All monthly salaries strictly greater than \$50,000. The final result table must report all three categories, even if a category contains zero accounts.

SOLUTION:



The screenshot shows the pgAdmin 4 interface with the following SQL queries and results:

```
1 CREATE TABLE Accounts (  
2     account_id INT PRIMARY KEY,  
3     income INT NOT NULL  
4 );  
5  
6 INSERT INTO Accounts (account_id, income) VALUES  
7 (3, 108939),  
8 (2, 12747),  
9 (8, 87709),  
10 (6, 91738),  
11 (7, 45169);  
12  
13  
14  
15 SELECT 'LOW CATEGORY' AS CATEGORY,  
16     COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT  
17 FROM ACCOUNTS  
18 WHERE INCOME <20000  
19 UNION ALL  
20 SELECT 'AVERAGE CATEGORY' AS CATEGORY,  
21     COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT  
22 FROM ACCOUNTS  
23 WHERE INCOME >20000 AND INCOME <=50000  
24 UNION ALL  
25 SELECT 'HIGH CATEGORY' AS CATEGORY,  
26     COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT  
27 FROM ACCOUNTS  
28 WHERE INCOME >50000;  
29  
30  
31
```

Total rows: 3 Query complete 00:00:00.071 CRLF Ln 4, Col 1

EXPERIMENT 1

OUTPUT:

Data Output Messages Notifications			
			
			
			
	category text	account_count bigint	
1	LOW CATEGORY	1	
2	AVERAGE CATEGORY	1	
3	HIGH CATEGORY	3	

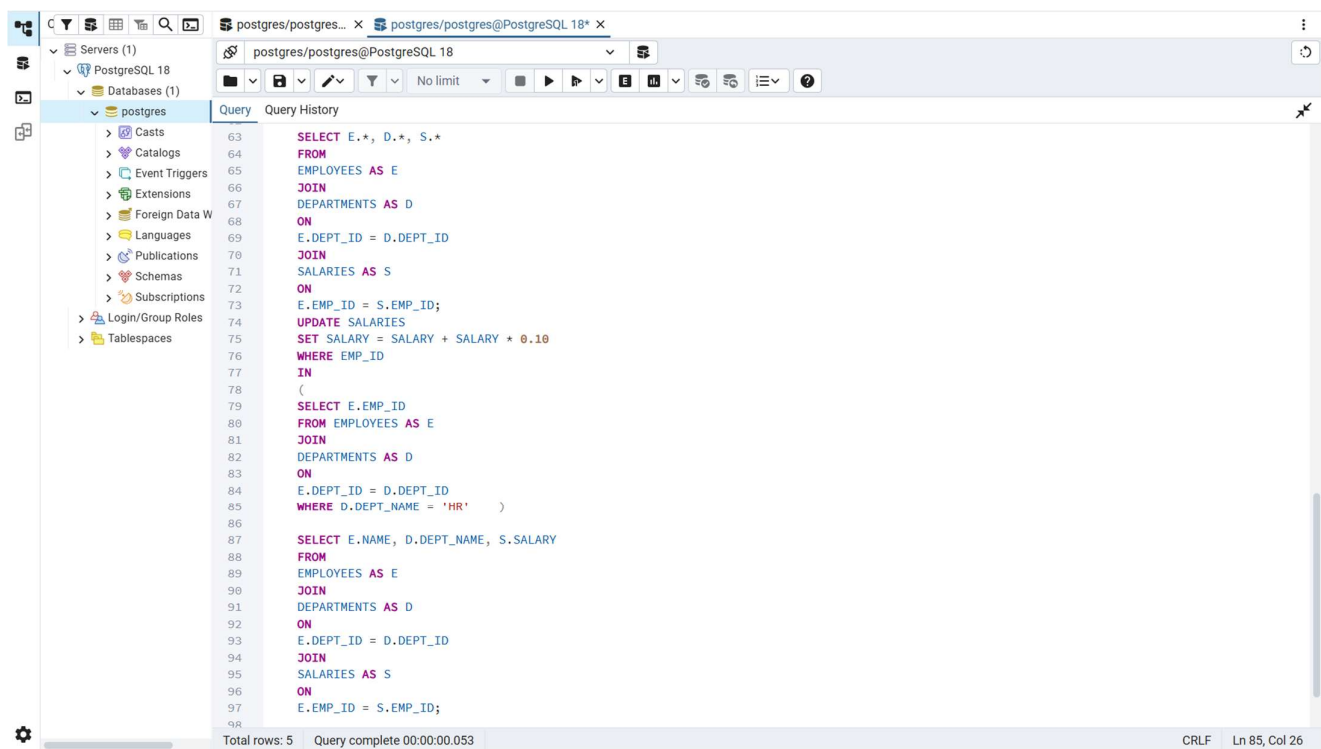
EXPERIMENT 1

QUESTION (B)

Design an Employee Management System database architecture by implementing the following operations sequentially:

1. Create and populate structural tracking tables for Departments, Employees, and Salaries.
2. Query the system using multi-table Joins to pull clear department breakdowns of employees, department and salaries, apply a 10% salary enhancement across all members of the HR department, and delete any individual salary record dropping strictly below \$30,000.
3. List all remaining active employees whose personal salary exceeds the overall company-wide average salary.

SOLUTION:



The screenshot shows a PostgreSQL query editor with a multi-step SQL script. The script is divided into three main sections, each starting with a line number (63, 74, 87). The first section (lines 63-73) performs a multi-table join query on EMPLOYEES, DEPARTMENTS, and SALARIES. The second section (lines 74-86) updates the SALARIES table by increasing the salary by 10% for employees in the HR department. The third section (lines 87-97) performs another multi-table join query to list the names, department names, and salaries of all employees. The status bar at the bottom indicates 'Total rows: 5' and 'Query complete 00:00:00.053'.

```
63 SELECT E.*, D.*, S.*
64 FROM
65 EMPLOYEES AS E
66 JOIN
67 DEPARTMENTS AS D
68 ON
69 E.DEPT_ID = D.DEPT_ID
70 JOIN
71 SALARIES AS S
72 ON
73 E.EMP_ID = S.EMP_ID;
74 UPDATE SALARIES
75 SET SALARY = SALARY + SALARY * 0.10
76 WHERE EMP_ID
77 IN
78 (
79 SELECT E.EMP_ID
80 FROM EMPLOYEES AS E
81 JOIN
82 DEPARTMENTS AS D
83 ON
84 E.DEPT_ID = D.DEPT_ID
85 WHERE D.DEPT_NAME = 'HR' )
86
87 SELECT E.NAME, D.DEPT_NAME, S.SALARY
88 FROM
89 EMPLOYEES AS E
90 JOIN
91 DEPARTMENTS AS D
92 ON
93 E.DEPT_ID = D.DEPT_ID
94 JOIN
95 SALARIES AS S
96 ON
97 E.EMP_ID = S.EMP_ID;
```

Total rows: 5 Query complete 00:00:00.053 CRLF Ln 85, Col 26

EXPERIMENT 1

OUTPUT:

Data Output Messages Notifications			
+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL			
	name character varying (50) 🔒	dept_name character varying (50) 🔒	salary integer 🔒
1	Charlie	IT	80000
2	David	IT	40000
3	Emma	Sales	45000
4	Alice	HR	55000
5	Bob	HR	27500

QUESTION (C):

A pizza restaurant stores all the available pizza toppings and their ingredient costs in a table called pizza_toppings.

Every customer must choose exactly 3 different toppings to prepare a pizza.

Write a PostgreSQL query to generate every possible 3-topping pizza combination along with its total ingredient cost.

Requirements

Every pizza must contain exactly 3 different toppings.

The same topping cannot appear more than once in a pizza.

Different arrangements of the same toppings should be considered the same pizza. Example:

Chicken, Pepperoni,
 Sausage Pepperoni,
 Chicken, Sausage
 Sausage, Chicken,
 Pepperoni

These are all the same pizza, so only one should appear.

Display the topping names separated by commas.

Display the total ingredient cost of each

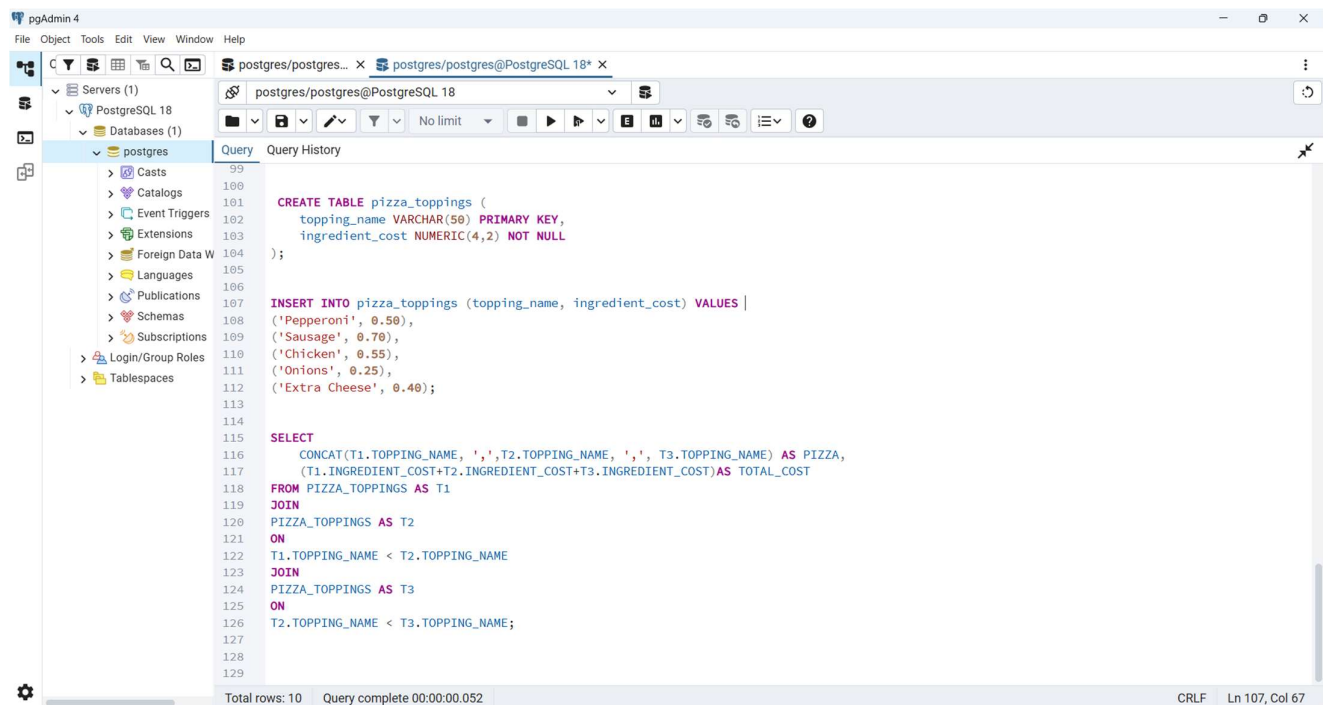
pizza. Sort the output:

First by highest total cost

If two pizzas have the same cost, sort them alphabetically.

EXPERIMENT 1

SOLUTION:



```

-- CREATE TABLE
CREATE TABLE pizza_toppings (
    topping_name VARCHAR(50) PRIMARY KEY,
    ingredient_cost NUMERIC(4,2) NOT NULL
);

-- INSERT INTO
INSERT INTO pizza_toppings (topping_name, ingredient_cost) VALUES (
    ('Pepperoni', 0.50),
    ('Sausage', 0.70),
    ('Chicken', 0.55),
    ('Onions', 0.25),
    ('Extra Cheese', 0.40);

-- SELECT
SELECT
    CONCAT(T1.TOPPING_NAME, ',', T2.TOPPING_NAME, ',', T3.TOPPING_NAME) AS PIZZA,
    (T1.INGREDIENT_COST+T2.INGREDIENT_COST+T3.INGREDIENT_COST) AS TOTAL_COST
FROM PIZZA_TOPPING AS T1
JOIN
    PIZZA_TOPPING AS T2
ON
    T1.TOPPING_NAME < T2.TOPPING_NAME
JOIN
    PIZZA_TOPPING AS T3
ON
    T2.TOPPING_NAME < T3.TOPPING_NAME;
    
```

Total rows: 10 Query complete 00:00:00.052 CRLF Ln 107, Col 67

OUTPUT:

	pizza text	total_cost numeric
1	Chicken,Extra Cheese,Onions	1.20
2	Chicken,Extra Cheese,Peppero...	1.45
3	Chicken,Extra Cheese,Sausage	1.65
4	Chicken,Onions,Pepperoni	1.30
5	Chicken,Onions,Sausage	1.50
6	Chicken,Pepperoni,Sausage	1.75
7	Onions,Pepperoni,Sausage	1.45
8	Extra Cheese,Onions,Pepperoni	1.15
9	Extra Cheese,Onions,Sausage	1.35
10	Extra Cheese,Pepperoni,Sausa...	1.60

EXPERIMENT 1

QUESTION (D)

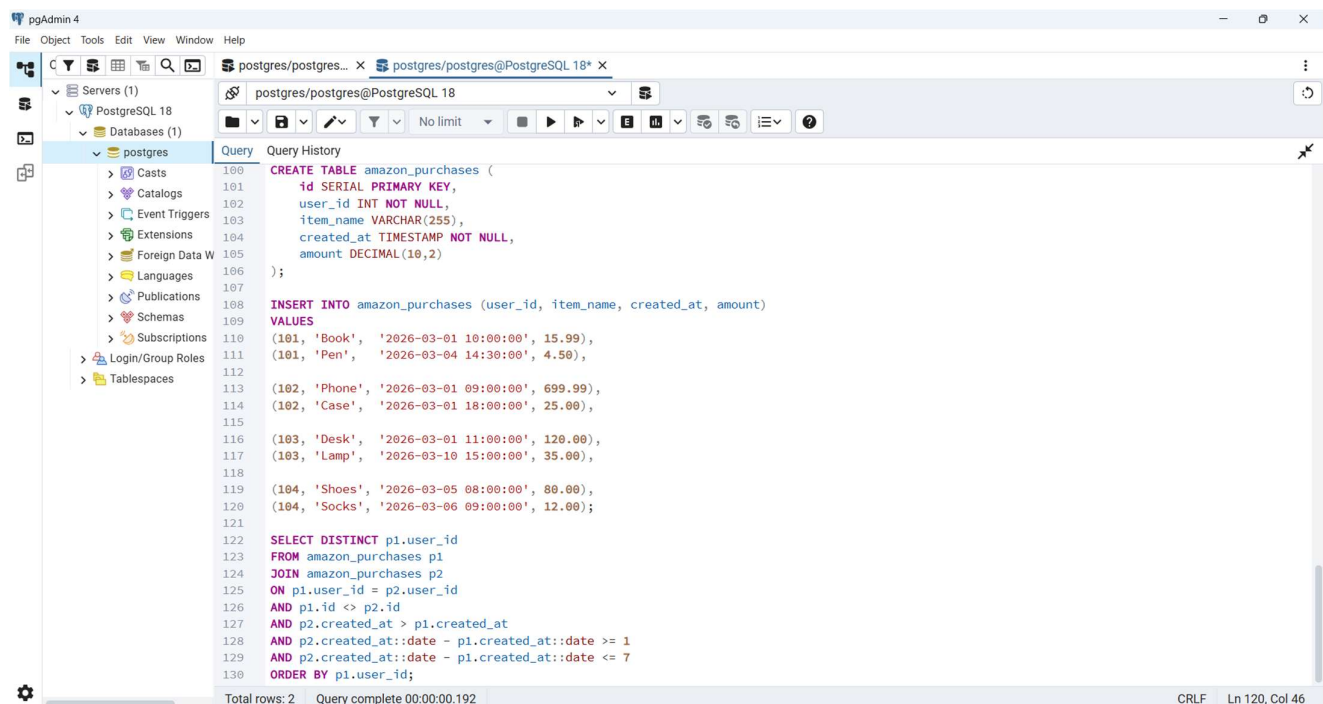
Amazon wants to identify returning customers who made another purchase shortly after their first purchase.

Write a PostgreSQL query to find all users who made another purchase within 1 to 7 days of an earlier purchase.

Requirements

1. Compare purchases made by the same user only.
2. Ignore comparisons where the same transaction is matched with itself.
3. The second purchase must occur after the first purchase.
4. Same-day purchases must not be considered.
5. The second purchase must occur within 7 days of the first purchase.
6. Display only the unique User IDs that satisfy the above conditions.
7. Sort the output in ascending order of user_id.

SOLUTION:



The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure for 'postgres' on a 'PostgreSQL 18' server. The main pane shows a SQL query being executed. The query creates a table 'amazon_purchases' with columns 'id', 'user_id', 'item_name', 'created_at', and 'amount'. It then inserts 10 rows of data representing purchases. Finally, it executes a query to find distinct user IDs who have made a second purchase within 1 to 7 days of their first purchase, excluding self-comparisons.

```
CREATE TABLE amazon_purchases (
    id SERIAL PRIMARY KEY,
    user_id INT NOT NULL,
    item_name VARCHAR(255),
    created_at TIMESTAMP NOT NULL,
    amount DECIMAL(10,2)
);

INSERT INTO amazon_purchases (user_id, item_name, created_at, amount)
VALUES
(101, 'Book', '2026-03-01 10:00:00', 15.99),
(101, 'Pen', '2026-03-04 14:30:00', 4.50),
(102, 'Phone', '2026-03-01 09:00:00', 699.99),
(102, 'Case', '2026-03-01 18:00:00', 25.00),
(103, 'Desk', '2026-03-01 11:00:00', 120.00),
(103, 'Lamp', '2026-03-10 15:00:00', 35.00),
(104, 'Shoes', '2026-03-05 08:00:00', 80.00),
(104, 'Socks', '2026-03-06 09:00:00', 12.00);

SELECT DISTINCT p1.user_id
FROM amazon_purchases p1
JOIN amazon_purchases p2
ON p1.user_id = p2.user_id
AND p1.id <> p2.id
AND p2.created_at > p1.created_at
AND p2.created_at::date - p1.created_at::date >= 1
AND p2.created_at::date - p1.created_at::date <= 7
ORDER BY p1.user_id;
```

Total rows: 2 Query complete 00:00:00.192 CRLF Ln 120, Col 46

EXPERIMENT 1

OUTPUT:

21 -- COUNT (ACCOUNT_ID) AS A

Data Output Messages Notifications

	user_id integer
1	101
2	104